

Construye una Calculadora — Mini Cajero

JavaScript · Node.js · Taller práctico

En este taller vas a unir todo lo que has aprendido —variables, tipos de datos, condicionales y bucles— en un solo programa: un mini cajero que hace operaciones una tras otra, como lo haría una app de pagos. Este es TU reto: la guía te acompaña con instrucciones y pistas, pero el código lo escribes tú.

Lo que vas a construir

Un cajero que recibe dos números y una operación (+, −, ×, ÷), calcula el resultado, y repite el proceso hasta que decidas cerrar la sesión.

Antes de empezar: piensa en esto

Piensa en una app como Nequi. Cuando alguien manda un pago, consulta un saldo o recibe plata... ¿la app hace una sola cuenta y se cierra? ¿O sigue atendiendo operación tras operación hasta que el usuario cierra sesión?

Paso 0 · Prepara tu proyecto

Vas a ejecutar tu código con Node.js desde la terminal de VS Code. Para leer lo que escribes por teclado, Node necesita un paquete llamado `prompt-sync` (el `prompt()` del navegador no existe en Node). Esta es la única parte donde te damos el código listo: es una herramienta nueva. De aquí en adelante, el código lo construyes tú.

1. Instala `prompt-sync` (una sola vez)

Abre la terminal de VS Code dentro de la carpeta de tu proyecto y ejecuta estos dos comandos:

 En la terminal:

```
npm init -y
npm install prompt-sync
```

El primero crea el archivo `package.json` en tu carpeta. El segundo descarga el paquete.

2. Aprende a usar `prompt-sync`

Crea un archivo nuevo llamado `cajero.js` en la misma carpeta. Así se importa el paquete y así se pide un dato por teclado (este es el patrón que vas a reutilizar):

 Ejemplo de uso — `cajero.js`

```
const prompt = require('prompt-sync')();

let nombre = prompt("¿Cuál es tu nombre? ");
console.log("Hola, " + nombre + "!");
```

Para ejecutarlo, en la terminal escribe `node cajero.js`. La función `prompt()` muestra el mensaje, espera a que escribas y presiones Enter, y **devuelve lo que escribiste**. Recuerda ese detalle: te va a servir en la Parte 1.

Pista

La primera línea (`const prompt = require('prompt-sync')();`) va SIEMPRE al inicio de tu

archivo. Sin ella, `prompt()` no existe. De aquí en adelante ya no mostramos código: usa este ejemplo como molde.

Parte 1 · Una sola operación

Antes de repetir, haz que funcione una vez. Primero lo simple, luego lo evolucionas. A partir de aquí, sigue los pasos y escribe tú el código.

Pide los datos

Necesitas guardar tres datos que la persona escribe. Siguiendo el molde del ejemplo anterior:

1. Crea una variable **numero1** que guarde lo que la persona escriba como primer número.
2. Crea una variable **operacion** que guarde el símbolo de la operación (+, -, *, /).
3. Crea una variable **numero2** que guarde el segundo número.

Muestra la suma (primer intento)

1. Crea una variable **resultado** que guarde la suma de **numero1** y **numero2**.
2. Muéstrala en pantalla con `console.log()`.

Prueba esto y observa

Ejecuta `node cajero.js` y escribe 5, luego +, luego 3. ¿El resultado es el que esperabas? Algo raro pasa. Obsérvalo con atención antes de seguir.

Para pensar

¿Por qué el resultado no es 8? Recuerda qué tipo de dato devuelve `prompt()`.

Arréglalo

Para que la operación sea numérica y no de texto, necesitas convertir lo que devuelve `prompt()` a número antes de operar. Investiga y usa la función `Number()` al capturar **numero1** y **numero2**.

Para pensar

Convertiste **numero1** y **numero2**. ¿Por qué NO conviene convertir **operacion** con `Number()`?

Parte 2 · Elegir la operación

Ya suma. Pero un cajero también resta, multiplica y divide. ¿Cómo decide cuál hacer?

Para pensar

Si la persona eligió – en vez de +, ¿qué estructura de las que ya conoces te permite ejecutar un cálculo distinto según lo que eligió?

Decide la operación

Usa una estructura condicional para revisar qué guardó la variable **operacion** y calcular según el caso:

- Si **operacion** es "+", el resultado es la suma.

- Si es "-", la resta. Si es "*", la multiplicación. Si es "/", la división.
- Agrega un caso final que atrape cualquier símbolo inválido y guarde un mensaje como "Operación no válida".

Al final, muestra resultado con `console.log()`, igual que antes.

Para pensar

¿Qué crees que pasaría si NO pusieras ese caso final para símbolos inválidos?

Un detalle curioso

Prueba dividir un número entre 0 y observa qué imprime. ¿Debería un cajero real permitir eso? Guarda la idea para los retos.

Parte 3 · Repetir como un cajero real

Ya hace una operación completa. Ahora, la parte que le da sentido al negocio.

Para pensar

Tu cajero hace UNA operación y se detiene. ¿Qué estructura te permite repetir el proceso muchas veces sin copiar y pegar el código?

Haz que se repita

Convierte tu cajero en uno que atienda varias operaciones seguidas:

1. Crea una variable de control, por ejemplo **activo**, que empiece en **true**.
2. Envuelve todo tu cajero (pedir datos + decidir operación + mostrar resultado) dentro de un bucle que se repita mientras **activo** sea **true**.
3. Al inicio de cada vuelta, pregunta si quiere hacer otra operación. Si responde "no", cambia **activo** a **false** para que el bucle termine.
4. Cuando el bucle termine, muestra un mensaje de despedida.

Para pensar

¿Qué pasaría si nunca cambiaras activo a false dentro del bucle?

Lo que acabas de lograr

Construiste un cajero que:

- usa variables para guardar los datos,
- maneja tipos de datos (texto que conviertes a número),
- decide con una estructura condicional, y
- repite con un bucle hasta cerrar sesión.

Eso es, en el fondo, lo que hace una app como Nequi cada día. Todo lo que aprendiste por separado, junto y con sentido.

¿Terminaste? Retos

Si ya tienes tu cajero funcionando, intenta estos usando lo que ya sabes:

- Validar división entre cero: antes de dividir, si numero2 es 0, muestra un mensaje en vez del resultado raro.
- Rechazar texto no numérico: si al convertir obtienes NaN, vuelve a pedir el número.
- Contador de operaciones: lleva la cuenta de cuántas operaciones hiciste antes de cerrar sesión, y muéstrala al final.